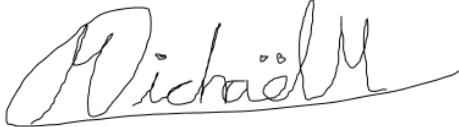


LAB 2: Dynamic Systems Modeling in Simulink

Course Number	MENG 3020
Course Title	System Modeling & Simulation
Semester/Year	Semester 3

Lab/Tutorial Report No.	Lab 2
Report Title	Dynamic Systems Modeling in Simulink
Section No.	0NA
Group No.	N/A
Submission Date	October 5 th , 2025
Due Date	October 5 th , 2025

Student Name	Signature*	Total Mark
Michaël McCorkell		/ 50

* By signing above, you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a ZERO on the work or possibly more severe penalties.

<https://academic-regulations.humber.ca/2021-2022/17.0-ACADEMIC-MISCONDUCT>

LAB 2 Grading Sheet

Student First Name: Michaël	Student Last Name: McCorkell
Part A.1: Nonlinear Model of Vehicle Suspension System	/15
Part A.2: Linear Model of Vehicle Suspension System	/8
Part A.3: Active Suspension Control System	/7
Post Lab Assignment	/15
General Formatting: Clarity, Writing style, Grammar, Spelling, Layout of the report	/5
Total Mark	/50

LAB 2: Dynamic Systems Modeling in Simulink

OBJECTIVES

- To learn how to simulate non-linear and linear differential equation model in Simulink.
- To compute the time response of a system to any arbitrary input signal.
- To explore modeling of equations of motion in Simulink.

INTRODUCTION

Simulink provides a graphical user interface that uses various types of elements called **blocks** to create a simulation of a dynamic system, which is a system that can be modeled with differential equations whose independent variable is time. For example, one block type is a *multiplier*, another performs a *sum*, and another is an *integrator*. Its graphical interface enables you to position the blocks, resize them, label them, specify block parameters, and interconnect the blocks to describe complicated system for simulation.

Type **simulink** in the MATLAB **Command** window to start **Simulink**. The Simulink window opens. To create a new model, click on the **Blank Model**. A new untiled window opens for you to create the model. Then open the **Library Browser** by selecting it in the drop-down menu under the **View** menu. To select a block from the **Library Broser**, click on the appropriate library category and a list of blocks within that category then appears. Click on the block name or icon, hold the mouse button down, drag the block to the new model window, and release the button. You can access help for that block by right-clicking on its name or icon and selecting **Help** from the drop-down menu.

Refer to **MENG-2000 Tutorial 2: Simulink Interactive Tutorial** document for more details on start **Simulink**.

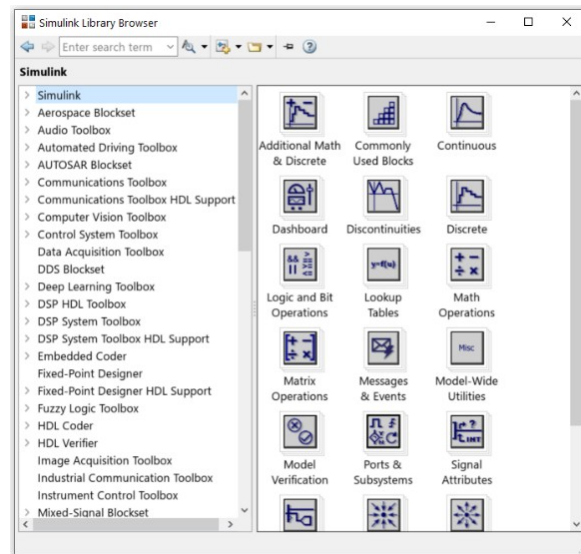


Figure 1: Simulink Library Browser

Simulink model files have the default extension **.slx** as of Release 2019a. The **.slx** format is a new compressed format that provides smaller file sizes. The **.mdl** format is still available as a save option.

You can use the **File** menu in the model window to *Open*, *Close*, and *Save* model files. To print the block diagram of the model, select **Print** on the **File** menu. Use the **Edit** menu to copy and paste blocks. You can also use the mouse for these operations. For example, to delete a block, click on it, and press the **Delete** key.

Simulink Model of Vehicle Suspension System

The **single-mass quarter-car model** of a vehicle suspension is shown below. In this simplified model, the masses of the wheel, tire, and axle are neglected, and the **mass** m represents one-fourth of the vehicle mass. The **spring constant** k models the series combination of the elasticity of the tire and the suspension spring. The **damping coefficient** b models the shock absorber. The motion of the mass m is produced by the motion $y(t)$ of the base, in this case, the road surface. The road surface displacement $y(t)$ can be derived from the road surface profile and the car's speed. The equilibrium position of mass m when $y=0$ is $x=0$.

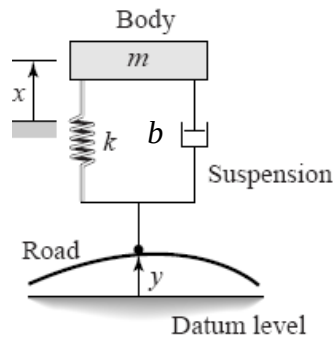


Figure 2: A quarter-car model with a single mass

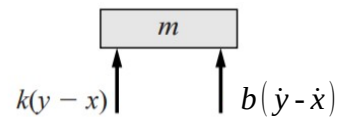
Linear Model & Transfer Function

Assume that the spring and damper have linear models, which means the **spring force** f_s is proportional to the relative displacement and the **damping force** f_d is proportional to the relative velocity.

$$f_s = k(y - x) \quad \text{and} \quad f_d = b(\dot{y} - \dot{x})$$

From the free body diagram, we obtain the **equation of motion**:

$$f_s + f_d = m\ddot{x} \quad \rightarrow \quad k(y - x) + b(\dot{y} - \dot{x}) = m\ddot{x}$$



The **transfer function model** is found by applying the Laplace transform to the equation, with the initial conditions set to zero:

$$\frac{Y(s)}{X(s)} = \frac{bs + k}{ms^2 + bs + k}$$

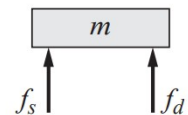
Nonlinear Model

In nonlinear spring and damper, there is no linear relationship with displacement and the velocity.

The **equation of motion** is in general form:

$$f_s + f_d = m\ddot{x}$$

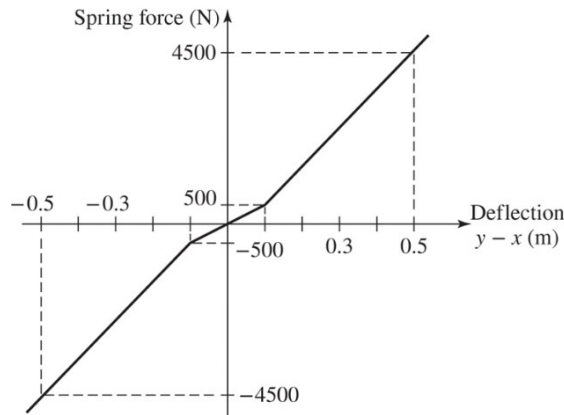
where, the **spring force** f_s and the **damping force** f_d are determined based on the nonlinear characteristics of the components.



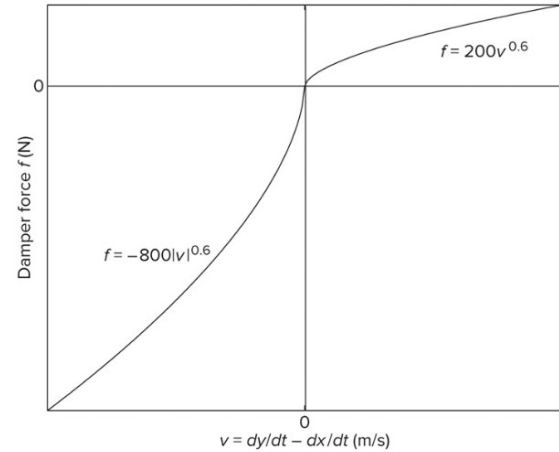
The goal is to derive the equation of motion of mass m with $y(t)$ as the input and obtain the Simulink model of the system.

A.1 Nonlinear Model of Vehicle Suspension System

In this part, assume that the **spring** and **damper** elements have the **nonlinear** models as shown below,



(a)



(b)

Figure 3: (a) Hardening Spring model, (b) Degressive Damper model

These models represent a **hardening spring** and a **degressive damper**. In addition, the damper model is *asymmetric*. It represents a damper whose damping during rebound is higher than during jounce (to minimize the force transmitted to the passenger compartment when the vehicle strikes a bump).

The bump is represented by the trapezoidal function $y(t)$ shown below. This function corresponds approximately to a vehicle traveling at 30 mi/h over a 0.2 m high road surface elevation 48 m long.

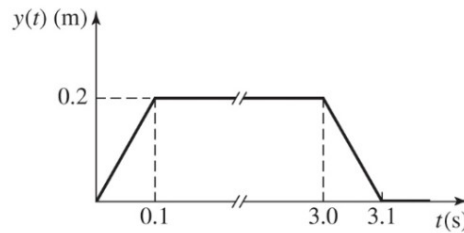


Figure 4: Road bump profile

The system model from Newton's law is:

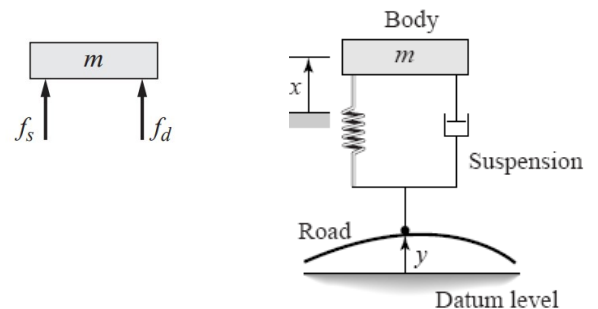
$$f_s + f_d = m\ddot{x}$$

where,

m is one-fourth of the vehicle mass ($m = 400$ kg),

f_s is the nonlinear spring function shown in Figure 3a,

f_d is the is the nonlinear damper function shown in Figure 3b,



x is the displacement of the car mass,

y is the displacement of road bump profile shown in Figure 4,

The corresponding simulation diagram is shown as:

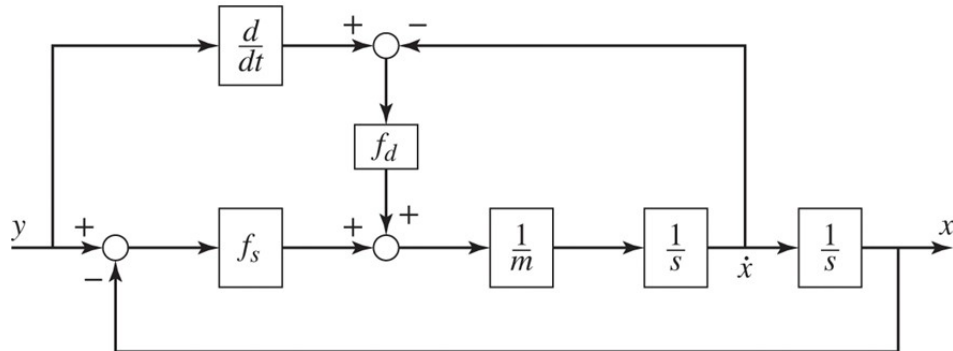


Figure 5: Block diagram model

This diagram shows we need to compute $\dot{y}$. Because Simulink uses numerical and not analytical methods, it computes derivatives only approximately, using the **Derivative** block. We must keep this in mind when using rapidly changing or discontinuous inputs.

1. Start **Simulink** and open a new window.
2. Create the following Simulink model to solve for and plot $x(t)$

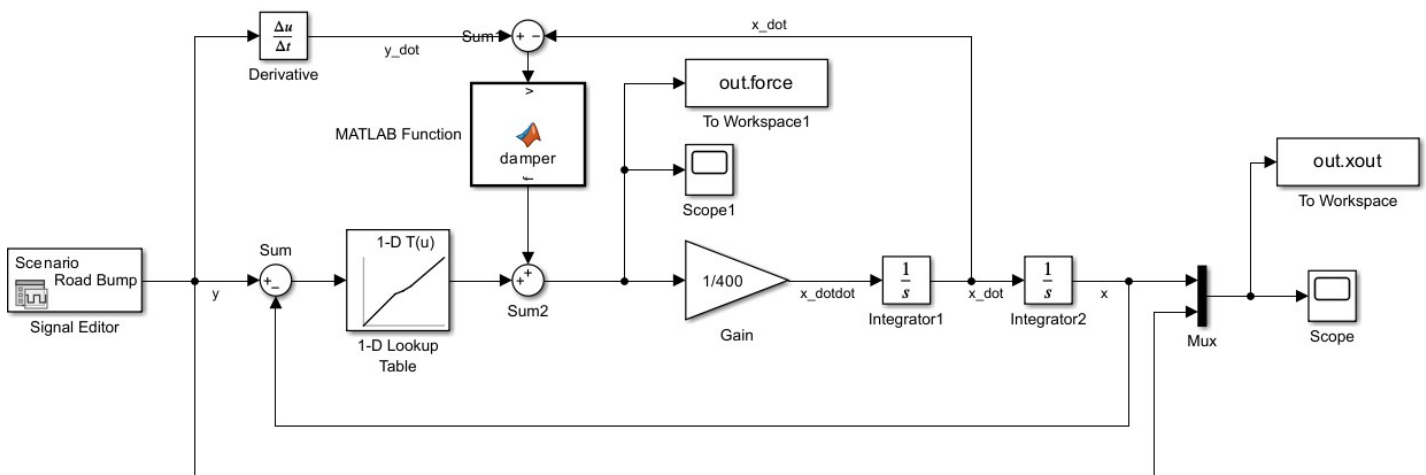


Figure 6: Simulink block diagram model

3. Open the **Simulink Library Browser** window and select the following blocks:

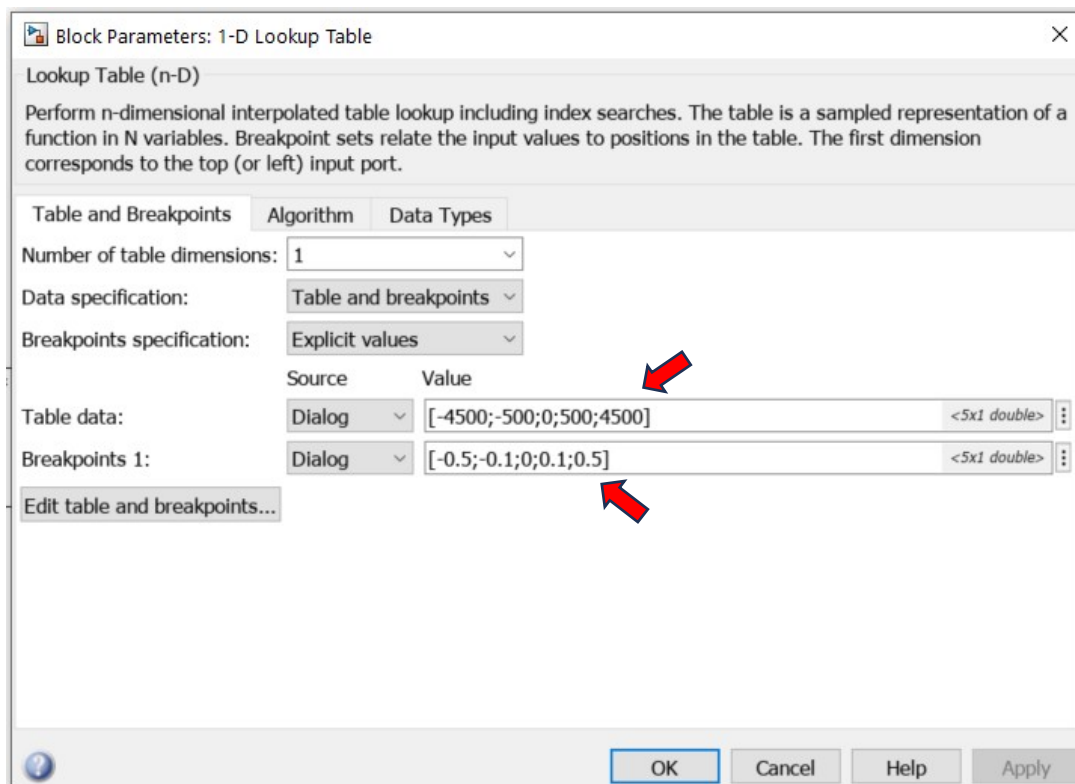
- The **Gain** block from the **Simulink/Math Operations** category.
 - Change the **Gain** value to $1/400$.
- The **Integrator** blocks from the **Simulink/Continuous** category.
- The **Derivative** block from the **Continuous** category.
- The **Mux** block from the **Signal Routing** category.
- The **Sum** blocks from the **Math Operations** category.
 - Set the signs as $|+-$, $++$ and $+-$. Rotate the blocks if required.

- Two **Scope** block from the **Simulink/Sink** category.
- Two **To Workspace** blocks from the **Simulink/Sink** category.
 - Change the output **Variable name** of **To Workspace** block to **xout** and specify the **Save Format** as **Array**.
 - Change the output **Variable name** of **To Workspace1** block to **force** and specify the **Save Format** as **Array**.
- The **MATLAB Function** block from **User-Defined Functions** category.
 - Open the **MATLAB Function** block and program the block as below:

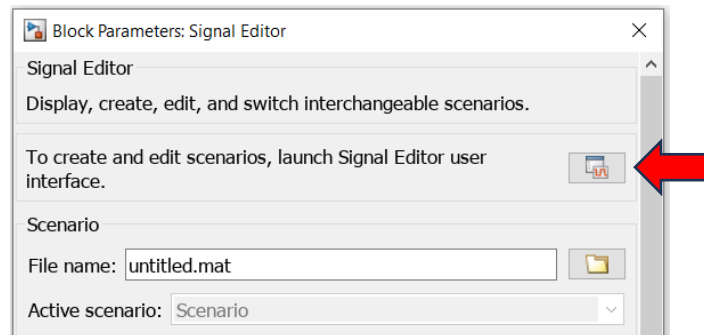
```
function f = damper(v)

if v <= 0
    f = -800*(abs(v)).^(0.6);
else
    f = 200*v.^(0.6);
end
```

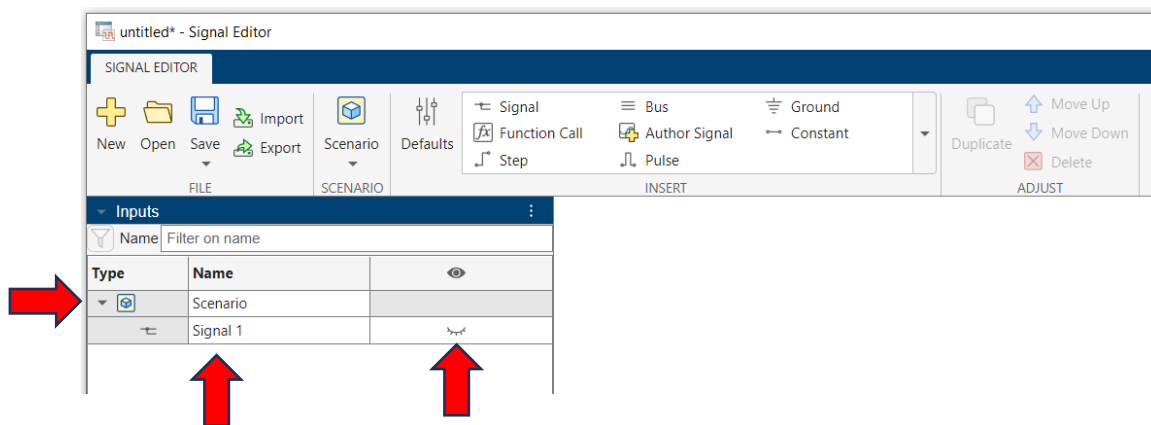
- The **1-D Lookup Table** block from **Lookup Tables** category.
 - Enter **[-4500 -500 0 500 4500]** for **Table data** values and **[-0.5 -0.1 0 0.1 0.5]** for **Breakpoints 1** values. Use default settings for the remaining parameters.



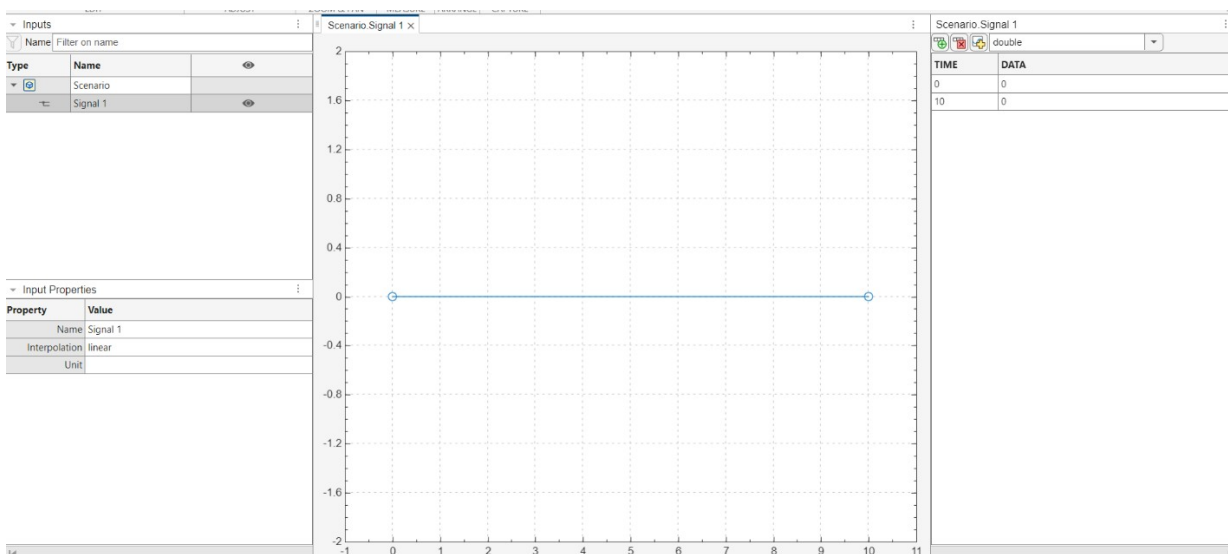
- The **Signal Editor** block from **Source** category.
 - Open the **Signal Editor** block to define the input function profile based on the Figure 6 as below:
 - Click on  icon to open the **Signal Editor** window.



- Click on the small drop-down icon beside the **Scenario** to see the signal present in it, which is named **Signal 1** by default.



- Double-click on the  icon in the right side of the **Signal 1** to reveal what is in this signal.



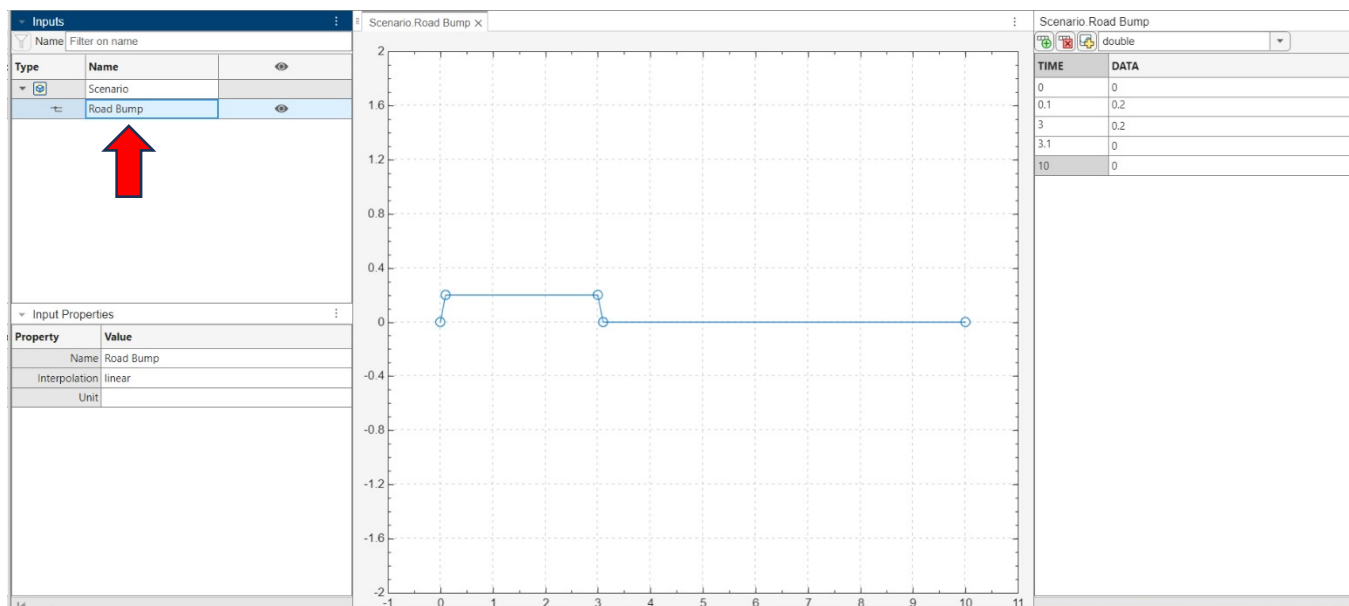
- The signal is blank at the first time. You can add the data for your signal profile by editing the data table in the right side.

Scenario.Signal 1	
double	
TIME	DATA
0	0
10	0

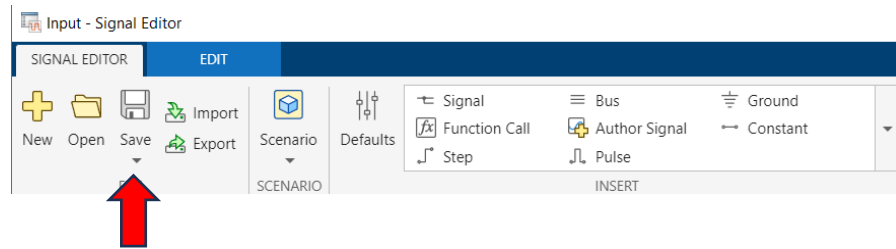
- Click on  icon to add more rows to the data table.
- Enter the **Time** values as [0 0.1 3 3.1 10] and **Data** values as [0 0.2 0.2 0 0] based on road bump profile from Figure 6.

Scenario.Signal 1	
double	
TIME	DATA
0	0
0.1	0.2
3	0.2
3.1	0
10	0

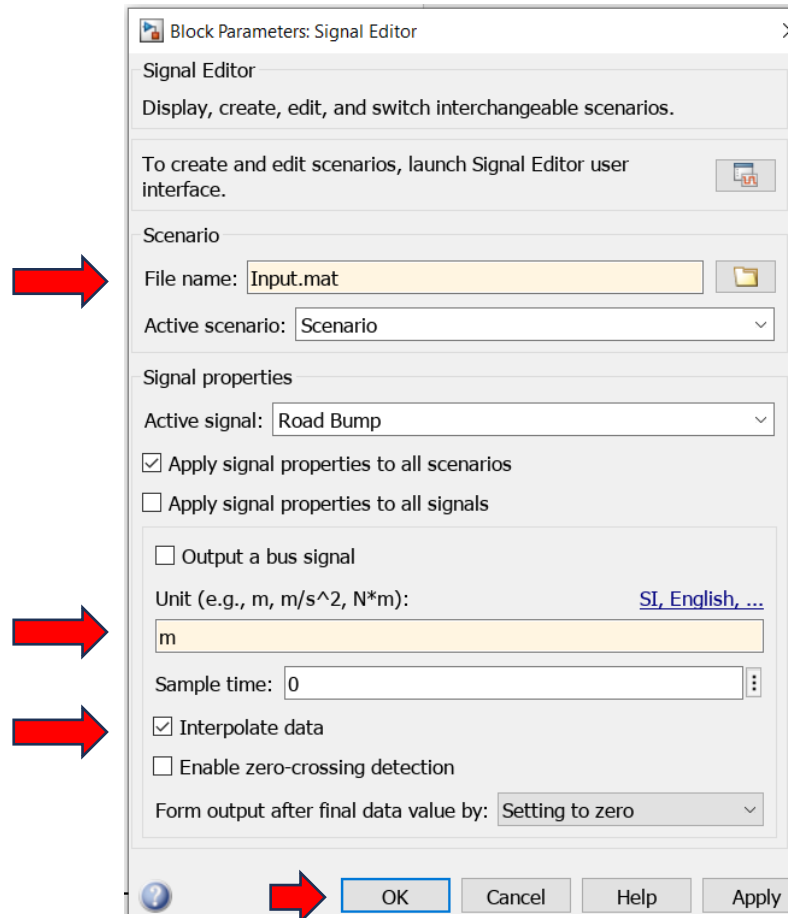
- The road bump profile will be created. Click on the signal name and rename the signal as **Road_Bump**.



- Save the signal file as **Input.mat** in the same folder you will save the Simulink file.



- The signal file name will be appeared in the **Block Parameters Dialogue** window.



- Set the signal data unit as **m** (meter) in the **Unit** box.
- Check marks the **Interpolate data** box.
- Click **OK** to close the dialogue window.

4. **Save** the Simulink model file as **Lab2_PartA1.slx**.

5. **Run** the simulation for **10 seconds**. Open the **Scope** block to check the results. Use **Cursor Measurement** to determine and record the required values in **Table 1**.

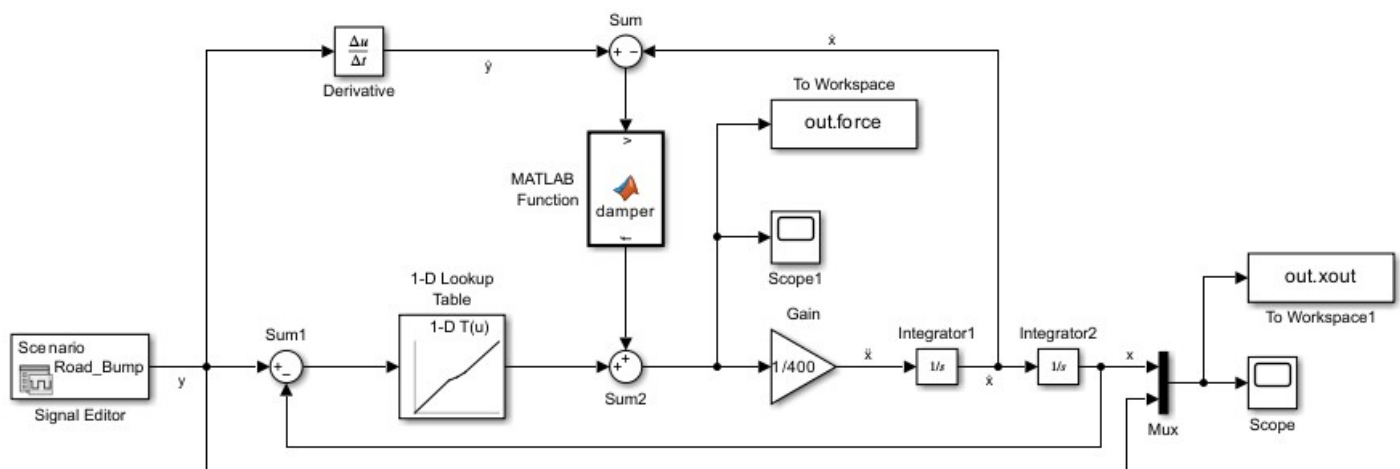
Table 1

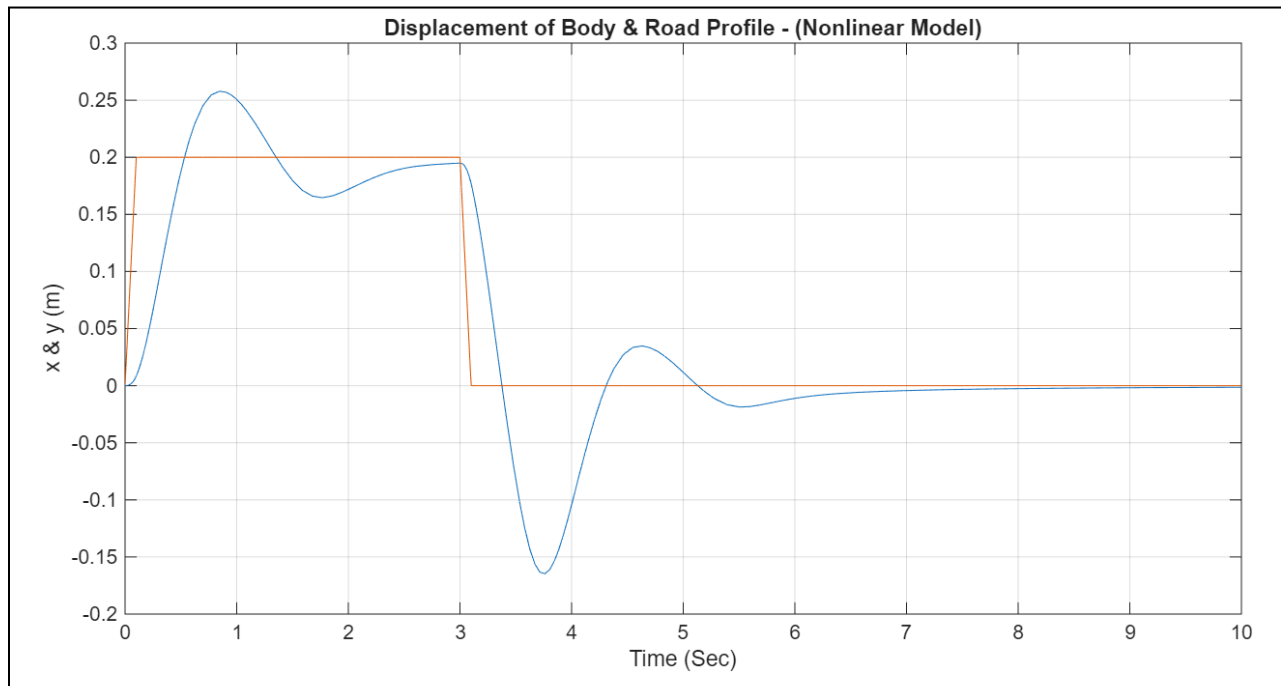
Maximum Overshoot (m)	Maximum Undershoot (m)
0.0580	0.1647

6. Plot the road bump profile and the displacement of body in MATLAB using the following code:

```
plot(out.tout,out.xout), grid on
xlabel('time (sec)'), ylabel('x & y (m)')
title('Displacement of Body & Road Profile - (Nonlinear Model)')
```

7. Provide your Simulink block diagram and the MATLAB plot below:

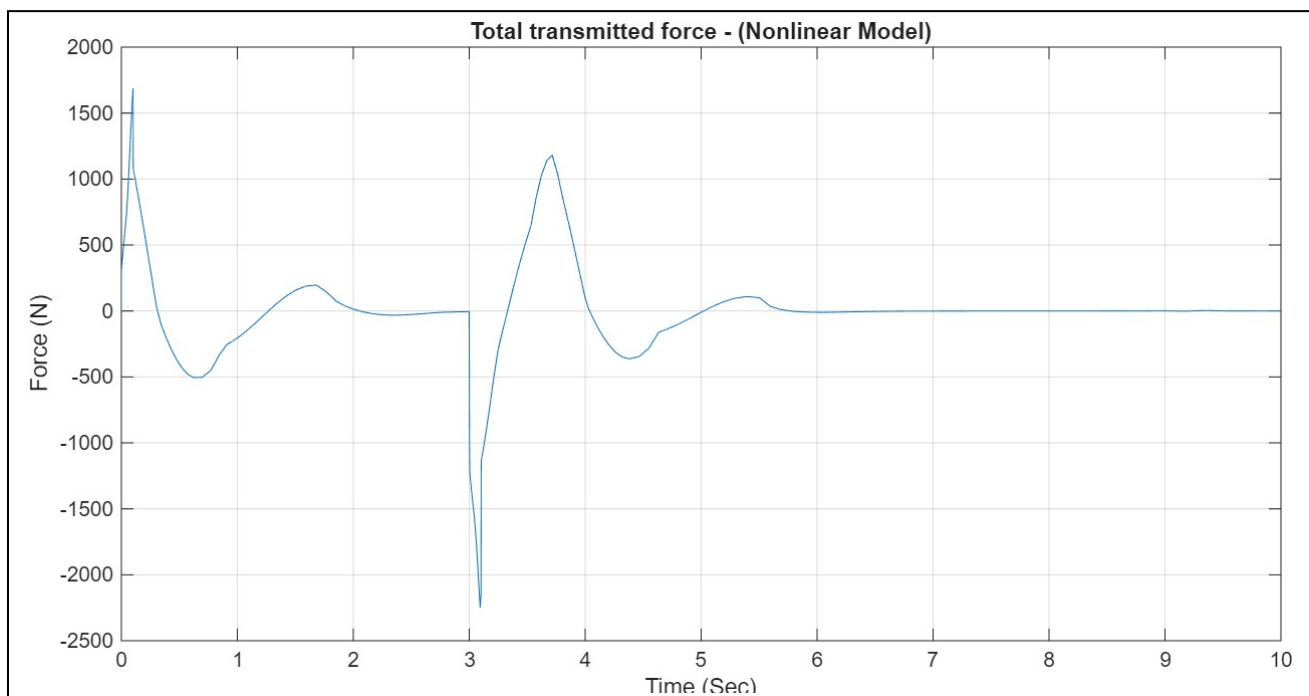




8. Open the **Scope 1** block to check the results, which displays the overall spring and damper forces transmitted to the chassis.
9. Plot the force graph in MATLAB using the following code:

```
plot(out.tout,out.force), grid on
xlabel('time (sec)'), ylabel('Force (N)')
title('Total transmitted force - (Nonlinear Model)')
```

10. Provide the force plot below:



A.2 Linear Model of Vehicle Suspension System

In this part, the goal is to derive a **linear model** for the spring and damper elements over a range of operation, where the displacement range is $-0.1 \leq y - x \leq 0.1$ m and the velocity range is $-0.1 \leq \dot{y} - \dot{x} \leq 0.1$ m/s.

The linearized **equation of motion** is:

$$f_s + f_d = m \ddot{x} \quad \rightarrow \quad k(y - x) + b(\dot{y} - \dot{x}) = m \ddot{x}$$

11. Estimate the **spring constant** k and **damping coefficient** b and obtain a linear approximation for each element. Show your work below.

$T = 3\text{sec} - 0.85\text{sec} \rightarrow 2.15 \text{ sec}$
 $\omega_d = 2\pi/T \rightarrow \approx 3 \text{ rad/s}$
 $p_1 = 0.25\text{m}$ $p_2 = 0.2\text{m}$
 $\log \text{ decrement} = \ln(p_1/p_2) \rightarrow \ln(0.25/0.2) \approx 0.22$
 $\text{dampening ration} = 0.22/\sqrt{4\pi^2 + 0.22^2} \approx 0.035$
 $\omega_n = \omega_d / \sqrt{1 - dr^2} \rightarrow 3/\sqrt{1 - 0.035^2} \rightarrow \approx 3.002 \text{ rad/s}$

 $k = m(\omega_n)^2 \rightarrow 400*(3.002)^2 \approx 3604.415 \text{ N/m}$
 $b = 2m(dr)(\omega_n) \rightarrow 2*400*0.035*3.002 \rightarrow 84.056 \text{ Ns/m}$

Provide the **linear model** of each element in **Table 2**.

Table 2

Linear Spring Model	Linear Damper Model
$k(y-x) \rightarrow 3604.415(y-x)$	$b(\dot{y}-\dot{x}) \rightarrow 84.056(\dot{y}-\dot{x})$

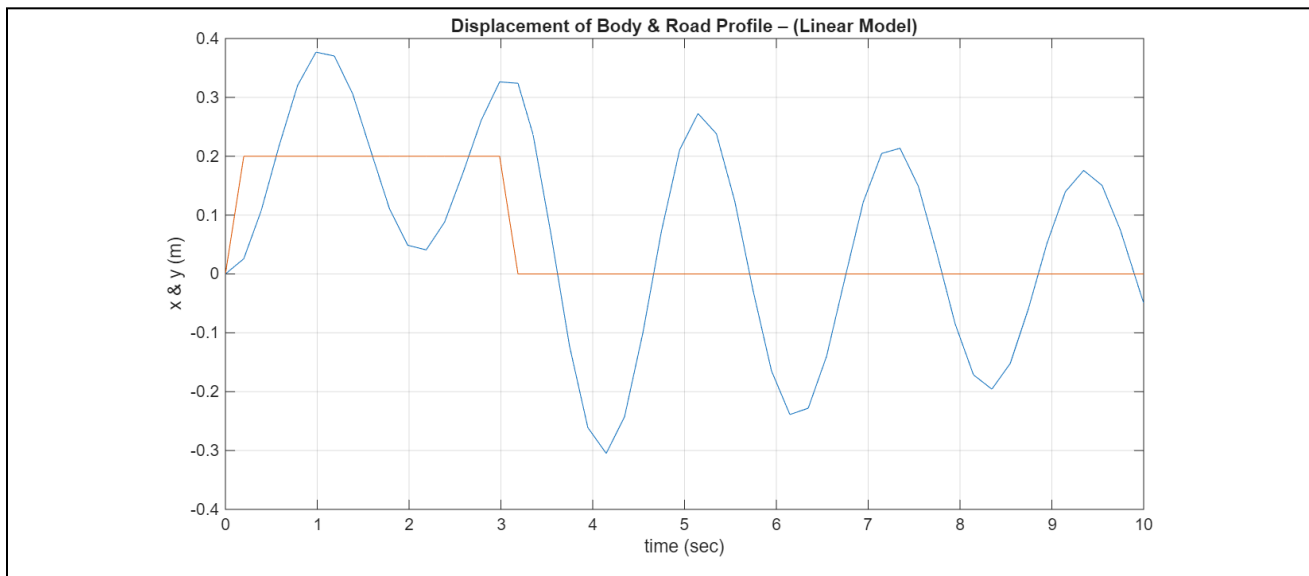
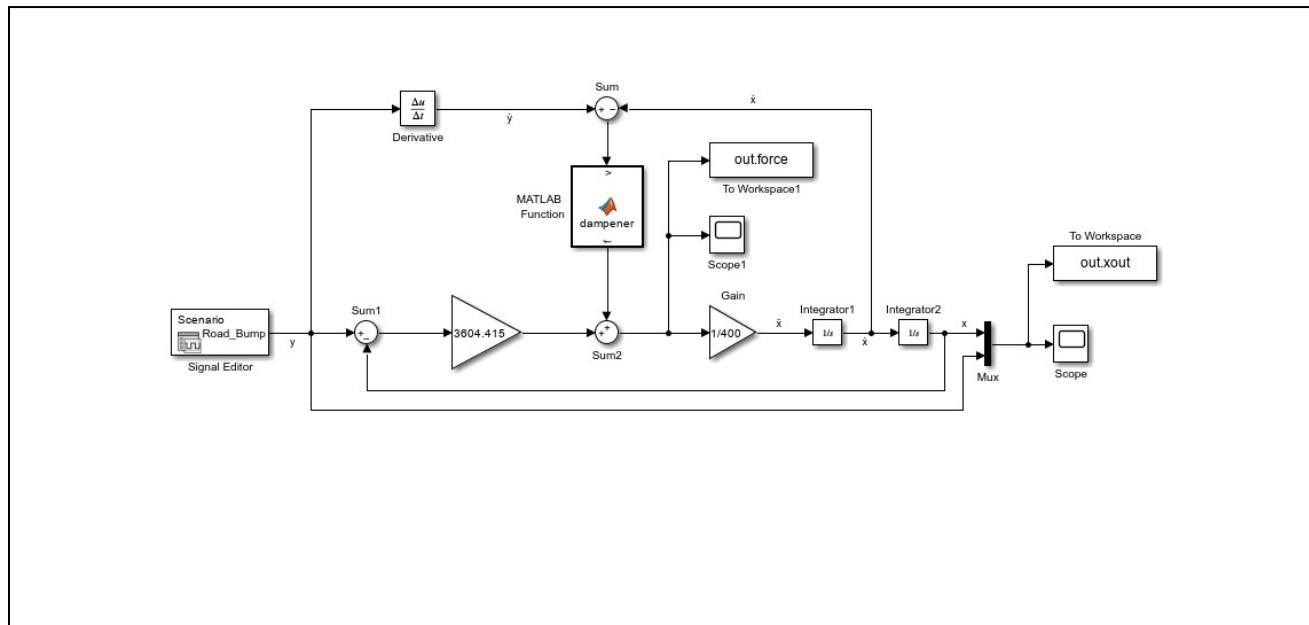
12. Modify the Simulink model diagram by replacing the **spring constant** block and editing **damping coefficient** block.
13. **Save** the Simulink model file as **Lab2_PartA2.slx**.
14. **Run** the simulation for **10 seconds**. Open the **Scope** block to check the results. Compare them with the obtained results of nonlinear model in **Part A1**. Is the linearized model acceptable in the given operation range?

Yes, the linearized model is acceptable. Since the linearized response closely matches the nonlinear model's steady or near-steady behavior (both showing little or no oscillation), it means the linear approximation accurately represents the system dynamics within that operating range.

15. Plot the road bump profile and the displacement of body in MATLAB using the following code:

```
plot(out.tout,out.xout), grid on
xlabel('time (sec)'), ylabel('x & y (m)')
title('Displacement of Body & Road Profile - (Linear Model)')
```

16. Provide your Simulink block diagram and the MATLAB plot below:



A.3 Active Suspension Control System

Oscillation of the body mass can be decreased by adding an **Active Suspension** controller to the system. An active suspension supplements the car's passive suspension system that consists of springs and shock absorbers. An electric motor or a hydraulic servomotor is mounted either in parallel or in series with the spring and shock absorber. The *actuator* is under computer control so that its force can be adjusted to apply more force in response to varying conditions, for example to maintain traction during turns or to compensate for increased passenger and cargo weight.

Figure 7 illustrates the principle for a single-mass, quarter-car suspension model. The **servomotor** is shown in parallel with the spring and damper. Assume that the **servomotor force** is proportional to the **chassis displacement** x and the **chassis velocity** $\dot{x}$, so that

$$f = -K_p x - K_D \dot{x}$$

where K_p and K_D are constants, whose value must be determined by the designer and programmed into the computer.

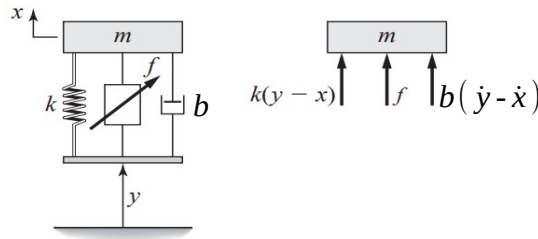


Figure 7: A quarter-car model with an active suspension system

Assuming the linearized model, the **equation of motion** is:

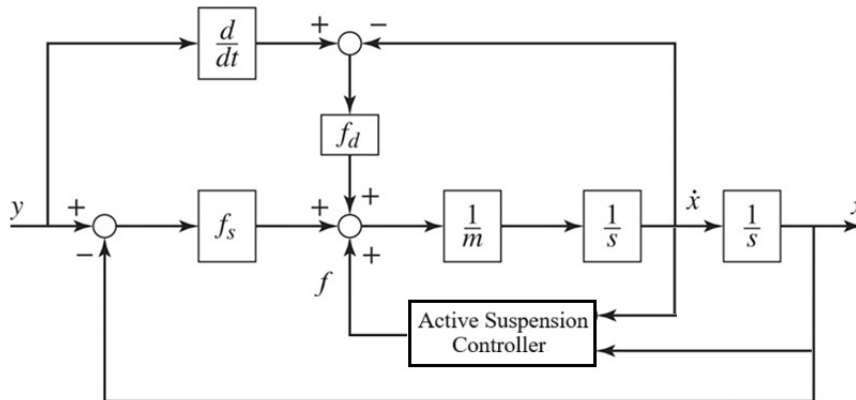
$$f_s + f_d + f = m \ddot{x} \quad \rightarrow \quad k(y - x) + b(\dot{y} - \dot{x}) + (-K_p x - K_D \dot{x}) = m \ddot{x}$$

Rearranging the equation:

$$m \ddot{x} + (b + K_D) \dot{x} + (k + K_p) x = b \dot{y} + ky$$

We immediately see that the gain K_p acts like **artificial stiffness**, and the gain K_D like an **artificial damper**. Thus, we can change the effectiveness stiffness and damping by properly selecting the gains.

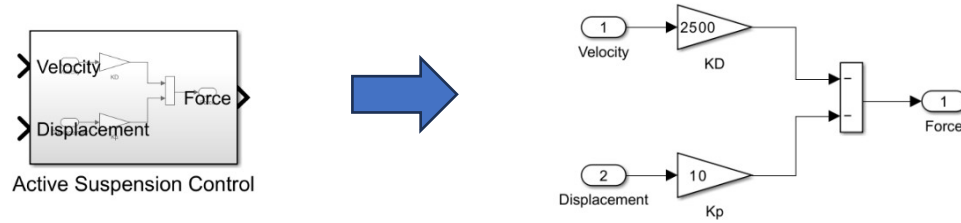
The corresponding simulation diagram is shown as:



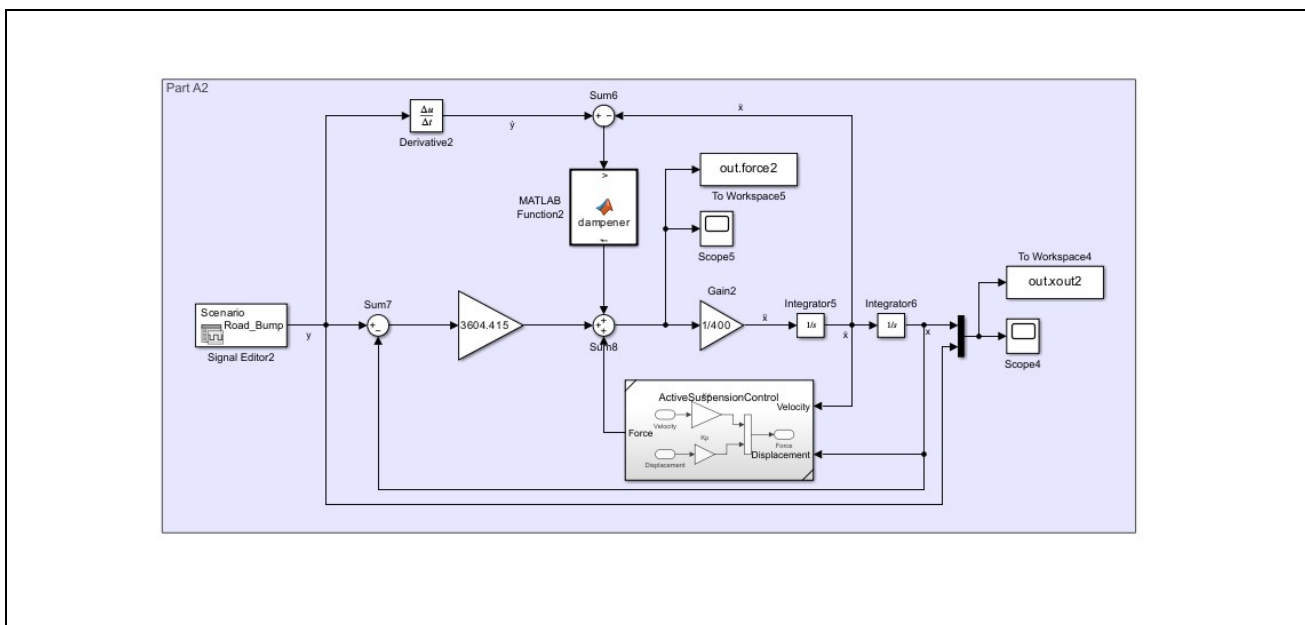
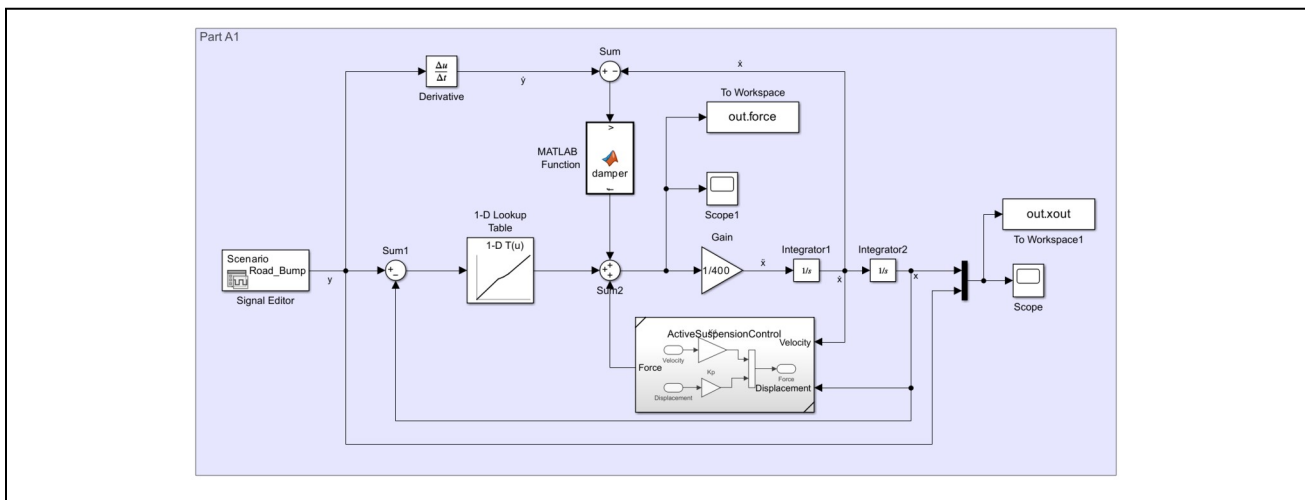
NOTE: Design of the controller gains is out of scope of this course. It is covered in the Control Systems course.

17. Select the controller gains as $K_p = 10$ and $K_D = 2500$.

18. Create the controller block using **Subsystem** block and including the required blocks as shown below



19. Modify the Simulink model diagram in **Part A1** and **Part A2** by adding the designed controller block. Provide the Simulink model diagrams below:



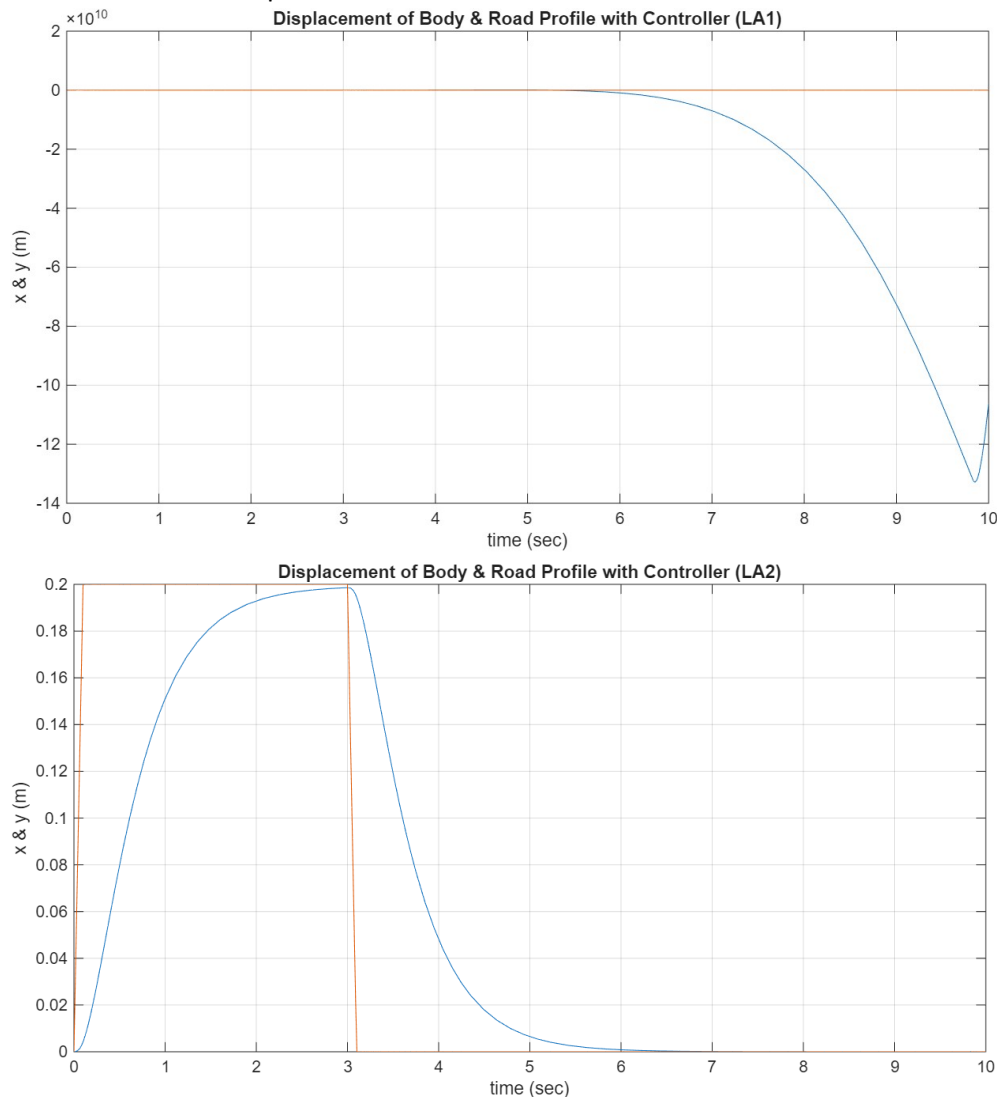
20. **Run** the simulation for **10 seconds**. Open the **Scope** block to check the results. Does the controller enhance the oscillation of the body mass?

No the controller does not enhance oscillations of the body mass. In the second plot, the yellow response shows a smooth rise and a well-damped return after the step input (blue), indicating that oscillations are reduced rather than amplified. The system settles quickly without sustained or increasing oscillations, meaning the controller effectively improves stability and damping.

21. Plot the road bump profile and the displacement of body in MATLAB using the following code:

```
plot(out.tout,out.xout), grid on
xlabel('time (sec)'), ylabel('x & y (m)')
title('Displacement of Body & Road Profile with Controller')
```

22. Provide the MATLAB plot below:



Post Lab Assignment

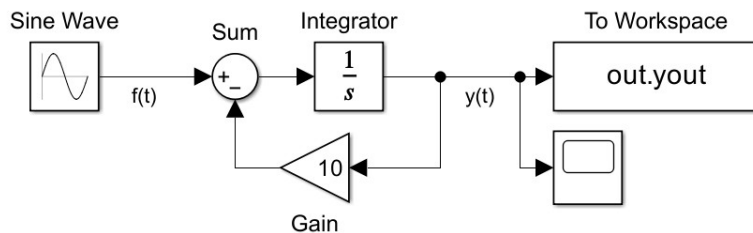
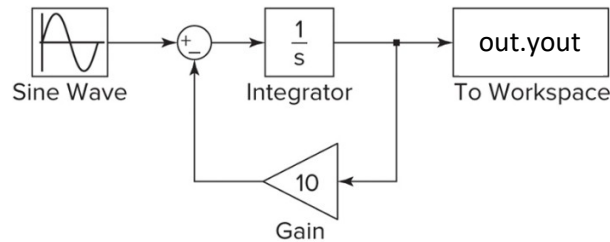
- 1) Develop a Simulink model to solve the following differential equation model:

$$\dot{y}(t) = -10y(t) + f(t), \quad y(0) = 1$$

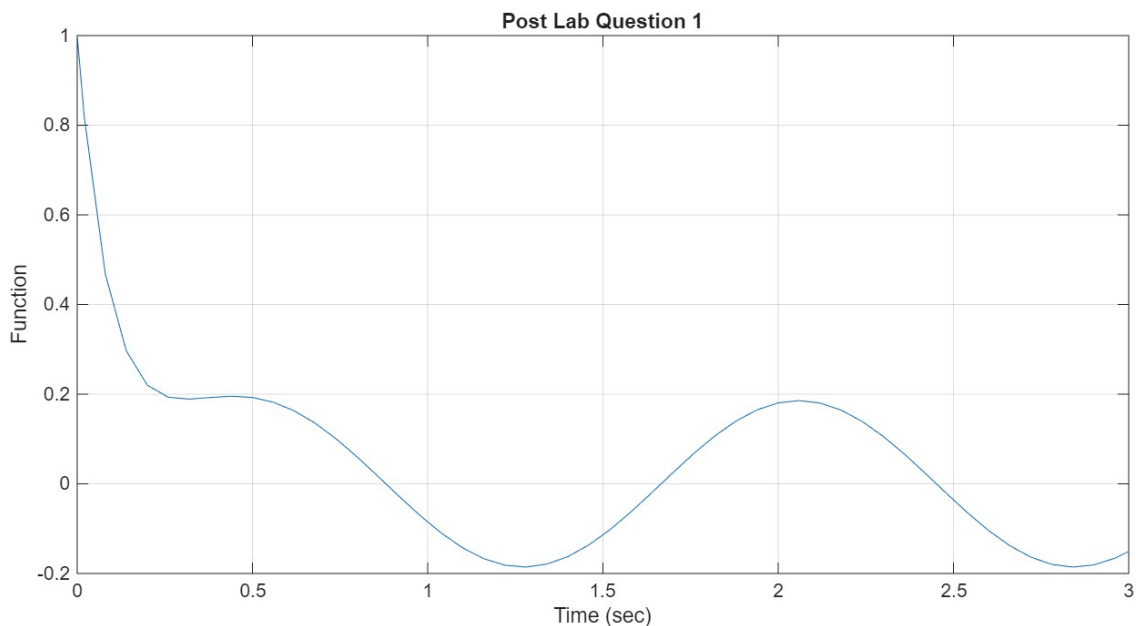
where $f(t) = 2 \sin 4t$, for $0 \leq t \leq 3$.

Run the simulation for **3 seconds** and plot the results in **MATLAB**. Provide the Simulink model, required code to plot the graph and the results. Add the required titles and labels to the graph.

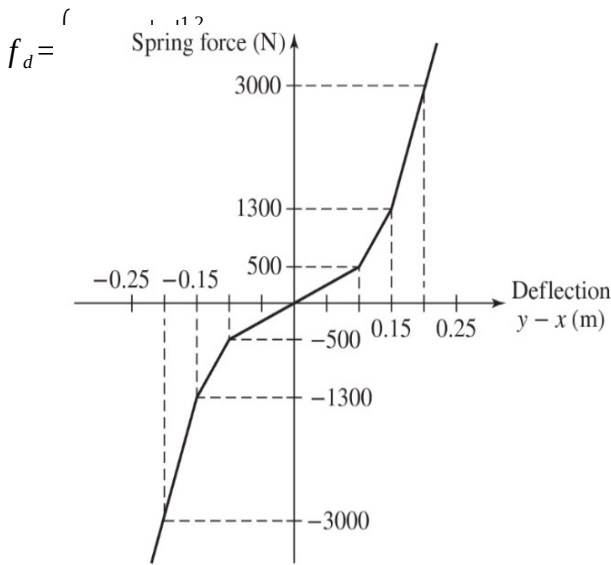
Hint: You can use the following model to develop the simulation.



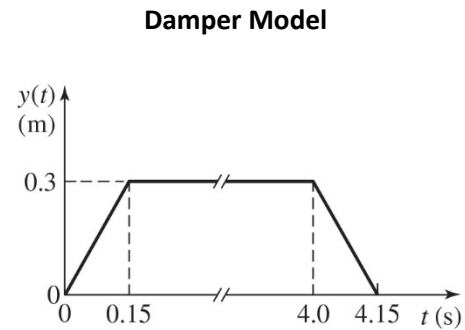
```
figure;
plot(out.yout), grid on
xlabel('Time (sec)'), ylabel('Function')
title('Post Lab Question 1')
```



- 2) Consider the Simulink suspension model developed in Part A.1 Figure 6. Modify the simulation using the following spring and damper relation and the given road bump profile as the input.



Spring Model



Road Bump Profile

Run the simulation for **15 seconds** and plot the responses in MATLAB. Provide the Simulink model, the required code to plot the graphs and the displacement graph and the force graph. Add the required titles and labels to the graphs.

Evaluate the overshoot and undershoot of the mass displacement. Provide the results.

Dampener function	<pre>function f = damper(v) if v<= 0 f = -500*(abs(v)).^(1.2); else f = 50*v.^(1.2); end</pre>
Spring force	<pre>[-3000;-1300;-500;0;500;1300;3000] [-0.2;-0.15;-0.1;0;0.1;0.15;0.2]</pre>
Code	<pre>plot(out.tout,out.xout), grid on xlabel('Time (Sec)'), ylabel('x & y (m)') title('Displacement of Body & Road Profile') figure; plot(out.tout,out.force), grid on xlabel('Time (Sec)'), ylabel('Force (N)') title('Total transmitted force')</pre>
Overshoot (m): 0.2246 m	Undershoot (m): 0.1814 m

